

EXPERIMENT #5&6

Convolution in Discrete Domain

Introduction:

Convolution is a mathematical operation equivalent to finite impulse response (FIR) filtering. It is a method of finding the zero-state response of relaxed linear time-invariant systems. It is important in digital signal processing because convolving two sequences in time domain is equivalent to multiplying the sequences in frequency domain. It is based on the concepts of linearity and time invariance and assumes that the system information is known in terms of impulse response. Correlation is a measure of similarity between two signals and is found using a process similar to convolution. There are two types of correlation: cross correlation and autocorrelation. In this chapter, various methods of evaluating the discrete convolution of finite duration sequences, and periodic sequences are discussed. The discrete cross correlation and autocorrelation and evaluation of these are also discussed.

In this lab, we will explore discrete-time convolution, in order to lay a better foundation for material to be presented later in the course. Convolution is an ubiquitous operation in signal processing, not least because it provides an elegant way to represent linear, time-invariant systems. The **convolution** of two signals x and y , in discrete time, is defined as

Equipment

For this lab, you will need a computer with Python 3 installed and the following libraries:

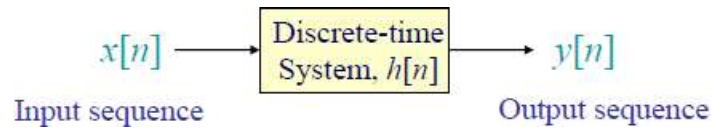
- Python
- numpy
- matplotlib

$$\begin{aligned}y[n] &= x[n] * h[n] \\&= \sum_{m=-\infty}^{\infty} x[m] \cdot h[n-m] \\&= \sum_{m=-\infty}^{\infty} h[m] \cdot x[n-m]\end{aligned}$$

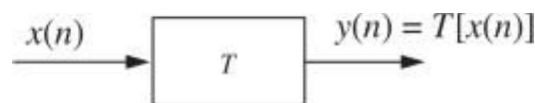
Where $h[n]$ is the impulse response of the system and it is at the heart of any DSP system.



Convolution in Discrete Domain



A discrete-time system performs an operation on an input signal based on a predefined criterion to produce a modified output signal. The input signal $x(n)$ is the system excitation and the output signal $y(n)$ is the system response. This transform operation is shown in Figure below



Convolution Process

The “ n ” dependency of $y[n]$ deserves some care: for each value of “ n ” the convolution sum must be computed *separately* over all values of a dummy variable “ m ”. So, for each “ n ”

1. Rename the independent variable as m . You now have $x[m]$ and $h[m]$. Flip $h[m]$ over the origin. This is $h[-m]$
2. Shift $h[-m]$ as far left as possible to a point “ n ”, where the two signals barely touch. This is $h[n-m]$
3. Multiply the two signals and sum over all values of m . This is the convolution sum for the specific “ n ” picked above.
4. Shift / move $h[-m]$ to the right by one sample, and obtain a new $h[n-m]$. Multiply and sum over all m .
5. Repeat 2~4 until $h[n-m]$ no longer overlaps with $x[m]$, i.e., shifted out of the $x[m]$ zone.

$$y[n] = x[n] * h[n] = \sum_{m=-\infty}^{\infty} x[m] \cdot h[n-m] = \sum_{m=-\infty}^{\infty} h[m] \cdot x[n-m]$$

Convolution using graphical method.



Convolution in Discrete Domain

EXAMPLE 2.11 Determine the convolution sum of two sequences:

$$x(n) = \{4, 2, 1, 3\}, \quad h(n) = \begin{Bmatrix} 1, 2, 2, 1 \\ \uparrow \end{Bmatrix}$$

Solution:

$x(n)$ starts at $n_1 = 0$ and $h(n)$ starts at $n_2 = -1$. Therefore, the starting sample of $y(n)$ is at

$$n = n_1 + n_2 = 0 - 1 = -1$$

$x(n)$ has 4 samples, $h(n)$ has 4 samples. Therefore, $y(n)$ will have $N = 4 + 4 - 1 = 7$ samples, i.e., from $n = -1$ to $n = 5$.

We know that

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$$

From Figure 2.6, we get

For $n = -1$ $y(-1) = \sum_{k=-\infty}^{\infty} x(k)h(-1-k) = 4 \cdot 1 = 4$

For $n = 0$ $y(0) = \sum_{k=-\infty}^{\infty} x(k)h(-k) = 4 \cdot 2 + 2 \cdot 1 = 10$

For $n = 1$ $y(1) = \sum_{k=-\infty}^{\infty} x(k)h(1-k) = 4 \cdot 2 + 2 \cdot 2 + 1 \cdot 1 = 13$

For $n = 2$ $y(2) = \sum_{k=-\infty}^{\infty} x(k)h(2-k) = 4 \cdot 1 + 2 \cdot 2 + 1 \cdot 2 + 3 \cdot 1 = 13$

For $n = 3$ $y(3) = \sum_{k=-\infty}^{\infty} x(k)h(3-k) = 2 \cdot 1 + 1 \cdot 2 + 3 \cdot 2 = 10$

For $n = 4$ $y(4) = \sum_{k=-\infty}^{\infty} x(k)h(4-k) = 1 \cdot 1 + 3 \cdot 2 = 7$



Convolution in Discrete Domain

Example: python has the built-in convolution function, *conv()* But be careful, however in setting the time axis

Procedure

Step 1: Import the required libraries

The first step is to import the required libraries. We will be using Numpy for numerical operations and matplotlib for plotting.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

Step 2: Define the signals

Next, we need to define the input signal and the impulse response. For this lab, we will use the following signals:

```
# Input signal
```

```
x = np.array([1, 2, 3, 4, 5])
```

```
# Impulse response
```

```
h = np.array([1, 1, 1])
```

Step 3: Implement the convolution function

Now, we can implement the convolution function. We will use the numpy convolve function to perform the convolution.

```
def convolve(x, h):
```

```
    y = np.convolve(x, h, mode='full')
```

```
    return y
```

Step 4: Perform the convolution

We can now perform the convolution by calling the convolve function.

```
y = convolve(x, h)
```

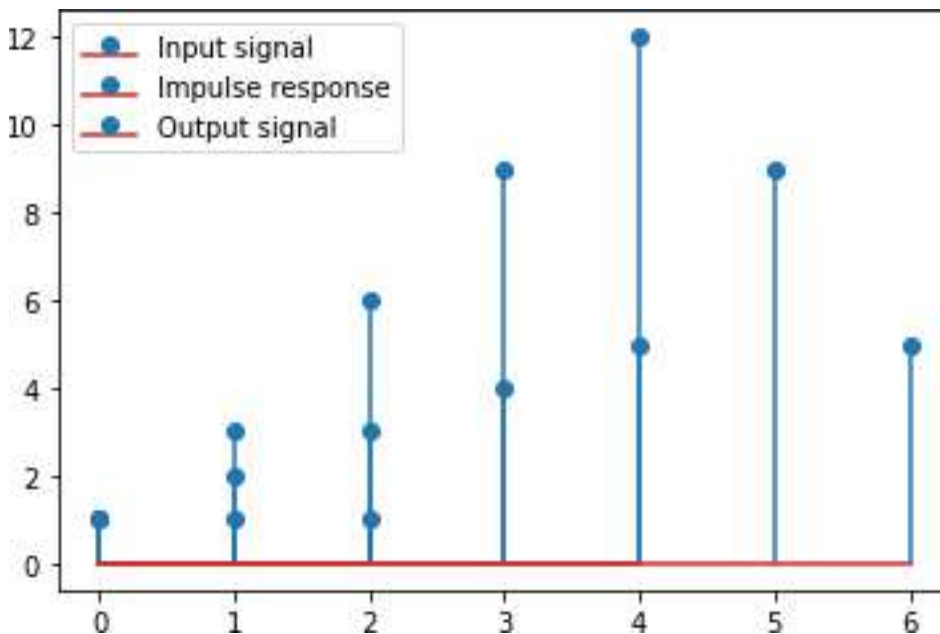
Step 5: Plot the signals



Convolution in Discrete Domain

Finally, we can plot the input signal, the impulse response, and the output signal.

```
plt.stem(x, label='Input signal')
plt.stem(h, label='Impulse response')
plt.stem(y, label='Output signal')
plt.legend()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

# Define the input signal and impulse response
x = np.array([1, 2, 4, 6, 8, 9, 10])
h = np.array([2, 1, 2])

# Define the convolution function
```



Convolution in Discrete Domain

```
def convolve(x, h):
```

```
    y = np.convolve(x, h)
```

```
    return y
```

```
# Compute the output signal by performing convolution
```

```
y = convolve(x, h)
```

```
# Plot the input signal, impulse response, and output signal using subplots
```

```
fig, axs = plt.subplots(3, 1, figsize=(8, 8), sharex=True)
```

```
axs[0].stem(x)
```

```
axs[0].set_ylabel('Amplitude')
```

```
axs[0].set_title('Input Signal')
```

```
axs[1].stem(h)
```

```
axs[1].set_ylabel('Amplitude')
```

```
axs[1].set_title('Impulse Response')
```

```
axs[2].stem(y)
```

```
axs[2].set_xlabel('Sample Index')
```

```
axs[2].set_ylabel('Amplitude')
```

```
axs[2].set_title('Output Signal')
```

```
plt.tight_layout()
```

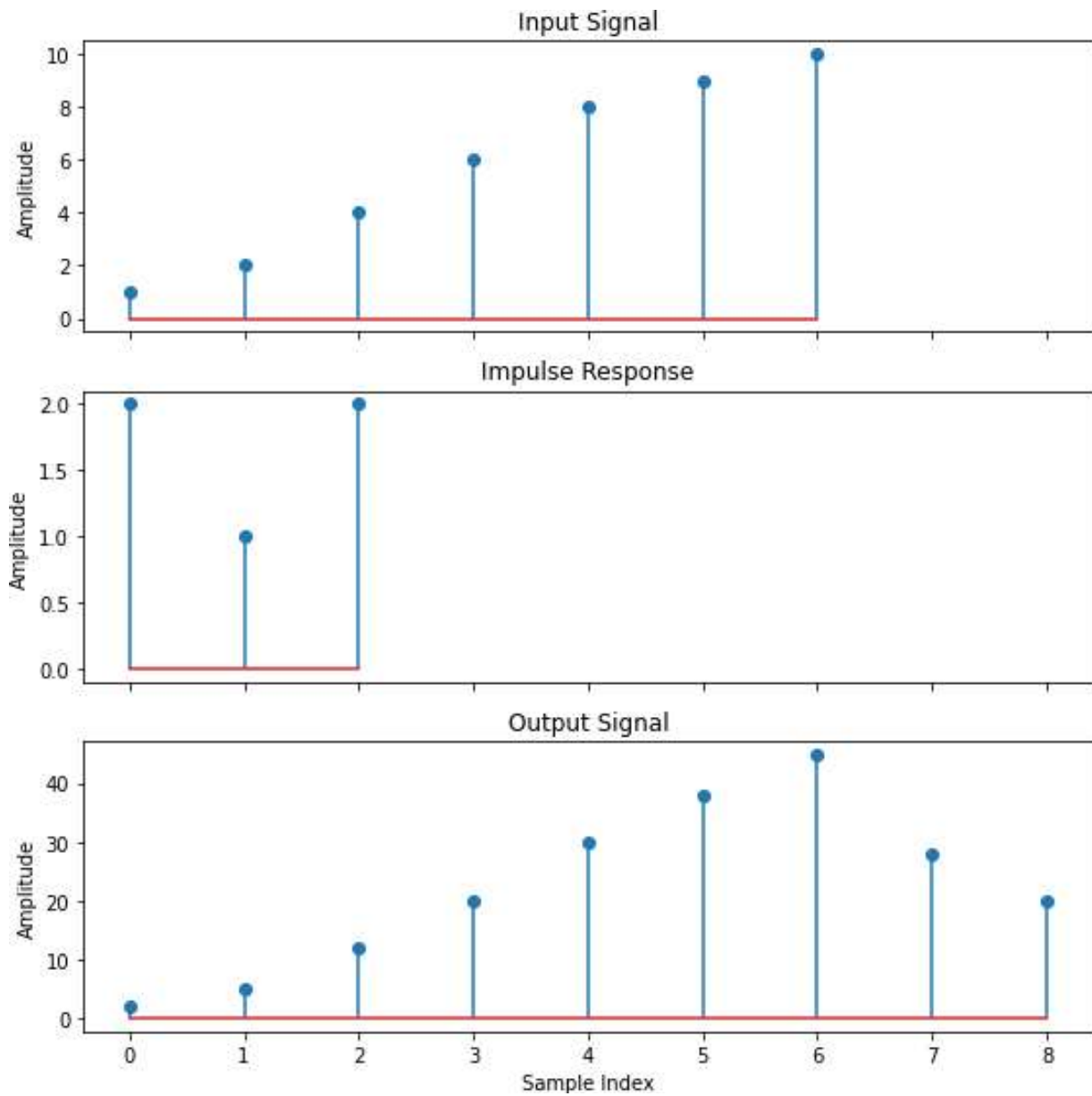
```
plt.show()
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER SCIENCE

Convolution in Discrete Domain





Convolution in Discrete Domain

Lab Tasks

Lab task example

Find the convolution of

$$x(n) = u(n),$$

$$h(n) = u(n - 3)$$

```
import numpy as np
```

```
# Define the signals
```

```
n = np.arange(0, 10) # Range of values for n
```

```
x = np.heaviside(n, 1) # x(n) = u(n)
```

```
h = np.heaviside(n-3, 1) # h(n) = u(n-3)
```

```
# Compute the convolution of x(n) and h(n)
```

```
y = np.convolve(x, h)
```

```
# Plot the input signals and the output signal
```

```
import matplotlib.pyplot as plt
```

```
plt.stem(n, x, use_line_collection=True)
```

```
plt.title('x(n) = u(n)')
```

```
plt.xlabel('n')
```

```
plt.ylabel('Amplitude')
```

```
plt.show()
```




Convolution in Discrete Domain

```
plt.stem(n, h, use_line_collection=True)
```

```
plt.title('h(n) = u(n-3)')
```

```
plt.xlabel('n')
```

```
plt.ylabel('Amplitude')
```

```
plt.show()
```

```
plt.stem(np.arange(0, 19), y, use_line_collection=True)
```

```
plt.title('y(n) = x(n) * h(n)')
```

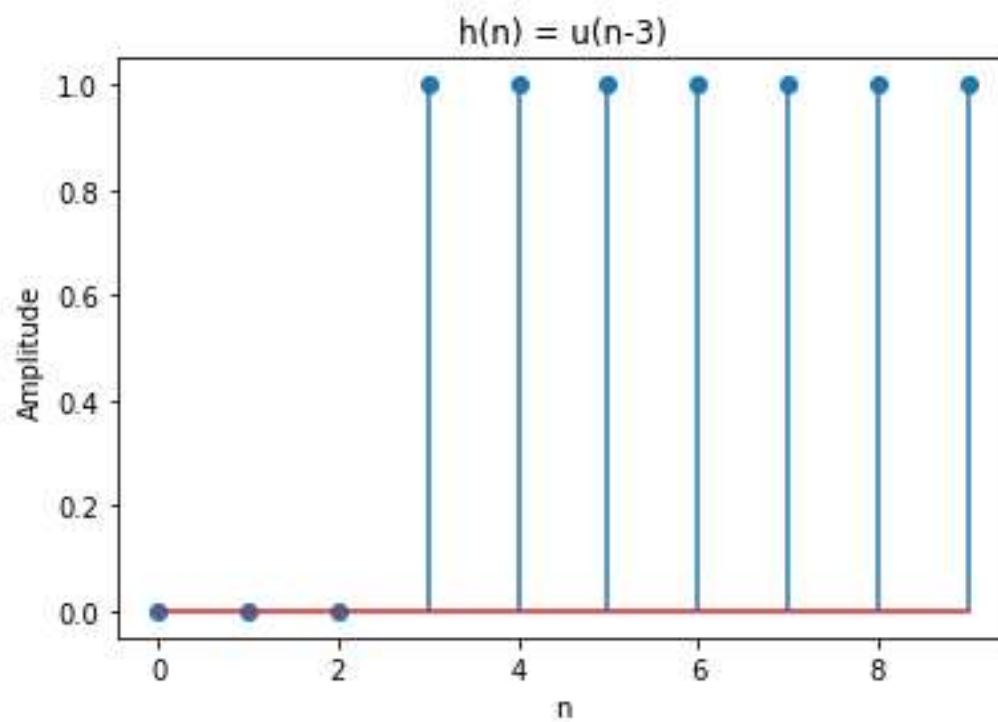
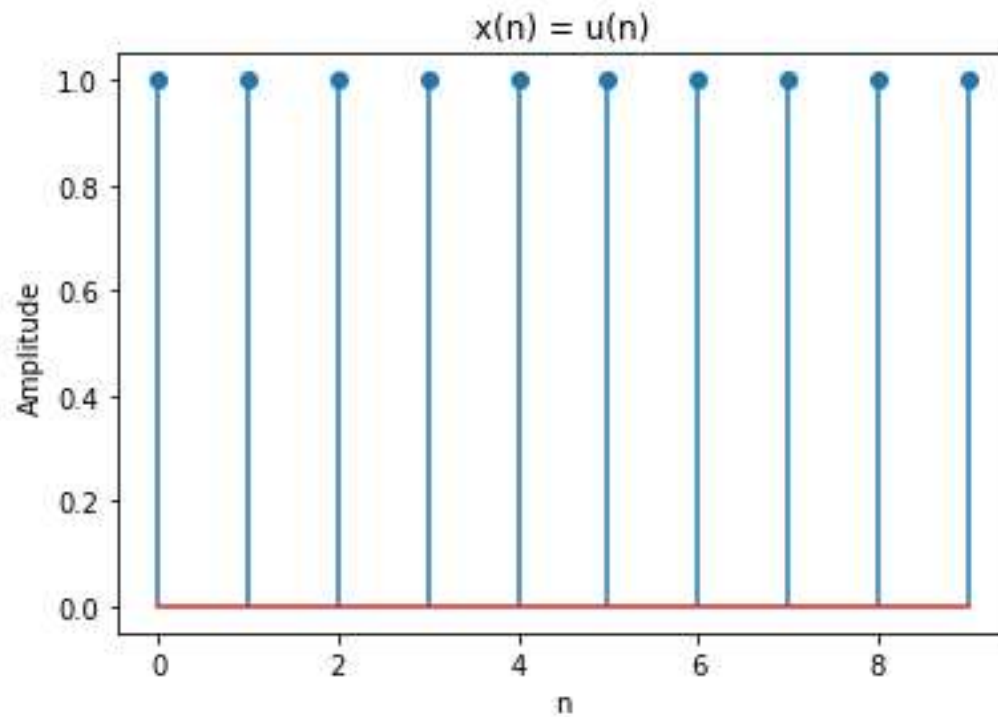
```
plt.xlabel('n')
```

```
plt.ylabel('Amplitude')
```

```
plt.show()
```

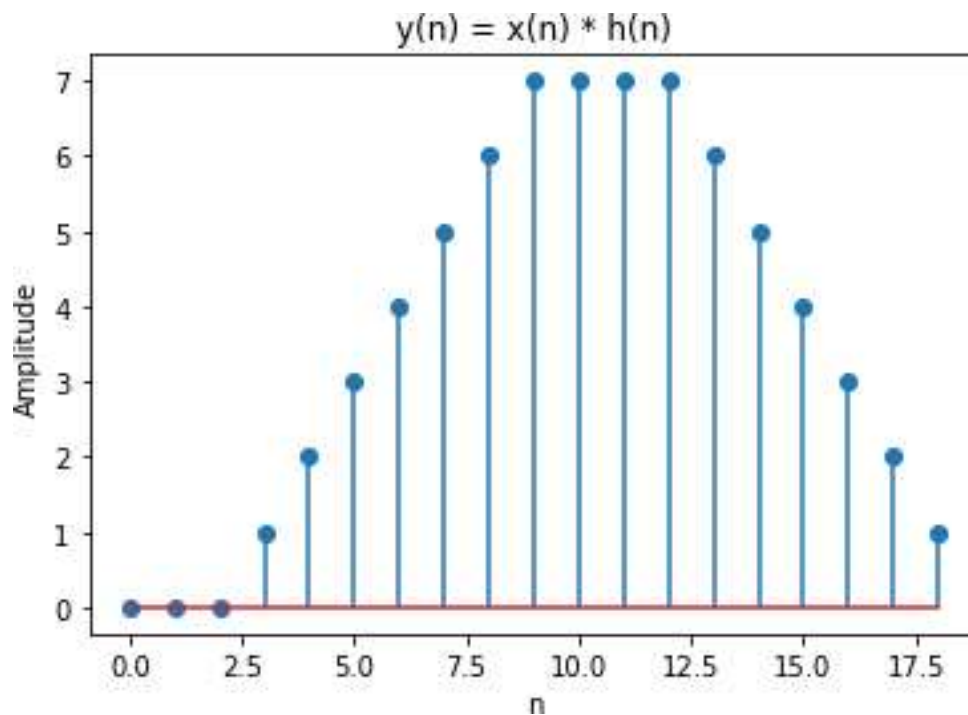


Convolution in Discrete Domain





Convolution in Discrete Domain



Task 1

Consider a system with unit sample response.

$$h(n) = u(n)a^n$$

Find the response to an input $x(n) = u(n) - u(n - N)$ find the convolution

Task 2

Find the convolution of

$$x(n) = (1/2)^n u(n),$$

$$h(n) = u(n) - u(n - 10)$$

Task 3

Find the convolution of



Convolution in Discrete Domain

$$x(n) = (1/3)^{-n} * u(-n-1),$$

$$h(n) = u(n - 1)$$

Task 4

Find the convolution of

$$x(n) = (1/3)^{-n} * u(-n-1),$$

$$h(n) = u(n - 1) - u(n - 2) + u(n + 3)$$

Task 5

$$(a) \ y(n) = \sin\left(\frac{n\pi}{2}\right) u(n) * u(n - 2) \quad (b) \ y(n) = 3^n u(-n + 3) * u(n - 2)$$

Task 5

$$y(n) = 3^n u(-n + 3) * u(n - 2)$$

$$x(n) = 3^n u(-n + 3)$$

$$h(n) = u(n - 2)$$

Conclusion